Objetivos para la entrega:

Desarrollar en los estudiantes las habilidades de la programación estructurada así como el uso de apuntadores, cadenas de caracteres, estructuras y archivos.

Problema: SISTEMA DE GESTIÓN DE UN RESTAURANTE

Un restaurante desea manejar la información sobre los diferentes menús que ofrece y los pedidos que realizan sus comensales. El sistema que se quiere construir debe permitir introducir diferentes ingredientes para los platos (cebolla, carne, pollo, tomate, gaseosa, pan, pasta, *etc*.), cada uno con un código, nombre, y precio de venta asociado por unidad; debe permitir también crear platos nuevos (por ejemplo, hamburguesa de pollo, tomate, y cebolla; bandeja paisa; lasaña; *etc.*) y establecer su precio total (con base en el precio y cantidad de cada uno de sus ingredientes), el cual debe incluir un cargo adicional por impuesto de IVA (19%). Para cada plato se debe establecer la cantidad exacta de cada ingrediente que se coloca en él. El conjunto de platos del restaurante con sus precios es lo que constituye la carta de menú que se ofrece a los comensales.

El sistema debe llevar el registro de las ventas a comensales, obtenidas por cada plato, y del inventario de ingredientes que tiene el restaurante.

Cuando un comensal realiza su pedido, se quiere actualizar el número de platos vendidos y los ingresos obtenidos por estos, y también actualizar el inventario, de manera que se pueda saber en todo momento la cantidad exacta de unidades que quedan de cada ingrediente (número de tomates, litros de gaseosa, libras de carne, *etc*.) para que se puedan realizar los pedidos a los proveedores del restaurante oportunamente. Adicionalmente, el sistema debería generar reportes diversos como los platos que más ingresos han generado, y los platos de los cuales se han vendido más unidades.

Algunas aclaraciones:

1. El Restaurante maneja un arreglo de ingredientes, un arreglo de platos y un arreglo de órdenes (de los comensales).
2. Para cada **`Ingrediente`** se registra el código (por ej: 10), nombre (por ej: "fríjoles rojos"), precio de la unidad (por ej: $2000), descripción de la unidad (por ej: "porción de 200 gr"), inventario actual (por ej: 50 unidades), mínimo valor del inventario que debería obligar a hacer un pedido al proveedor (por ejemplo, cuando se llegue a 10 unidades).
3. Un **`IngredientePlato`** indica en qué cantidad de unidades va un **`Ingrediente`** en un **`Plato`** Por ejemplo, del ingrediente "fríjoles rojos" se utilizan 2 porciones en el plato "bandeja paisa".
4. Todos los platos tienen un cargo adicional por impuesto de IVA del 19%.
5. Una **`OrdenPlato`** indica en qué cantidad va un Plato en una Orden de un comensal.
6. Cada **`Orden`** de un comensal incluye uno o varios platos (para él y para sus acompañantes). Para cada Orden se registra código y valor. El valor deberá calcularse a partir de los platos ordenados.

Se debe ofrecer un menú con las siguientes opciones:

```
RESTAURANTE JAVEREST
   opción 1:  agregar ingredientes al sistema
   opción 2:  agregar platos al sistema
   opción 3:  mostrar el menú del restaurante
   opción 4:  agregar una orden al sistema
   opción 5:  mostrar los tres platos más solicitados
   opción 6:  mostrar los tres platos más rentables
   opción 7: Salir
```

Se pide implementar las siguientes funcionalidades: (entre [] se muestra el valor en puntos para la evaluación)

1. **[15]** <u>Agregar ingredientes al sistema</u>:
   Si el usuario escoge esta opción, el sistema debe leer datos de cada ingrediente de un archivo de texto **cuyo nombre se le solicita al usuario** y agregarlos al Restaurante invocando una función llamada agregarIngrediente. Un ejemplo del contenido de este archivo es el siguiente (observe que, # indica una línea de comentario, * es separador dentro de una línea que describe un ingrediente y el código 0 indica fin de datos).

```
#código--nombre----precioUnitario---descripciónUnidad---inventario---mínimo
10 *frijoles rojos   *2000       *porción 200 gr        *50         *10
20 *arroz blanco     *1000       *libra                 *40         *10
30 *harina fina      *200        *libra                 *8          *3
40 *chicharrón seco  *1500       *porción 100 gr        *70         *15
50 *tomate maduro    *800        *libra                 *20         *5
60 *carne molida     *1000       *porción 300 gr        *40         *10
0
```

2. **[15]** Agregar platos al sistema:
   Si el usuario escoge esta opción, el sistema debe leer datos de cada plato de un archivo de texto **cuyo nombre se le solicita al usuario** y agregarlos al Restaurante invocando una función denominada agregarPlato. Un ejemplo del contenido de este archivo es el siguiente:

```
#PLATO
#código------nombre-------------
100          *bandeja Paisa
#ingredientes
10  2
20  1
40  2
0
#PLATO
#código------nombre-------------
200          *lasagna
#ingredientes
30  1
60  2
0
#PLATO
#código------nombre-------------
300         *pizza siciliana
#ingredientes
30  1
50  1
0
#FIN
```

   Aclaraciones:
   - Observe que para un Plato no se indica su precio, el cual posteriormente será calculado a partir de sus ingredientes; para un Plato se indican su código y nombre. El nuevo Plato debe quedar agregado al arreglo de platos.
   - Para cada Plato se indican sus ingredientes; para cada ingrediente se indica su código y la cantidad que va en el Plato. Con estos valores se deben crear todos los IngredientePlato relacionados con el Plato (invocando la función agregarIngredientePlato que debe ofrecer Restaurante).
   - Tenga en cuenta que un código 0 indica fin de datos de ingredientes de un plato y #FIN indica que no hay más platos; además # indica una línea de comentario y * es separador en una línea que describe un plato.

3. **[10]** Mostrar el menú del restaurante:
   Si el usuario escoge esta opción, el sistema debe mostrar para cada plato el código, nombre y precio, en donde el precio debe ser calculado teniendo en cuenta la definición de la función **calcularPrecio()** para un plato.

```
--MENU DEL RESTAURANTE: lista de platos ofrecidos:
código      nombre            precio
100         bandeja paisa     15310
200         lasagna           15500
300         pizza siciliana   20500
```

4. **[30]** Agregar una orden al sistema:
   Si el usuario escoge esta opción, el sistema debe proponer el código único para la nueva orden (a partir de un consecutivo). Luego debe preguntar al usuario cuántos tipos de platos componen la nueva orden y pedirle que ingrese el código y cantidad de unidades de cada tipo de plato que hace parte de la orden.

   Primero se debe validar que, si uno de los platos solicitados no se ofrece, entonces no se puede aceptar la orden. Se debe presentar un mensaje de error al usuario

   Segundo, se debe revisar el inventario de ingredientes para decidir si se puede o no satisfacer la orden. Vale la pena indicar que sólo se puede despachar una orden completa, es decir, que satisfaga todos los platos de la orden.

   Cuando se agrega una orden al sistema, es porque hay inventario de ingredientes suficientes para todos los platos de dicha orden. Si se puede agregar la orden al sistema, se debe reducir el inventario de los ingredientes.

   Si no hay inventario suficiente para satisfacer TODOS los Platos de una orden, NO se debe agregar dicha orden y no se afectará el inventario de Ingredientes.

   Al final debe informar el valor de la orden (aplicando el cálculo del precio de los distintos platos que componen la orden).

   Por ejemplo, para una orden compuesta de dos bandejas paisa y tres lasagnas, la interacción con el usuario sería así (suponiendo que se puede satisfacer la Orden):

```
--AGREGAR ORDEN: 1000
--¿Cuántos tipos de platos van en la orden?:
2
--Indique para cada tipo de plato: código  cantidad(de unidades/porciones)
100  2
200  3
El valor de la orden es $ 77120
```

5. **[15]** Mostrar los tres platos más solicitados:
   Si el usuario escoge esta opción, el sistema debe calcular, para cada plato del sistema, cuántas veces ha sido solicitado en todas las órdenes de los comensales y mostrar en orden descendente los tres platos con mejores resultados. Por ejemplo:

```
--PLATOS MÁS SOLICITADOS:
código   nombre            solicitudes
100      bandeja paisa     15
300      pizza siciliana   12
200      lasagna           10
```

6. **[15]** Mostrar los tres platos más rentables:
   Si el usuario escoge esta opción, el sistema debe calcular, para cada plato del sistema, cuál es el total de ventas en todas las órdenes de los comensales y mostrar en orden ascendente los tres platos con mejores resultados. Por ejemplo:

```
--PLATOS MAS RENTABLES:
codigo   nombre                 total de ventas
300      pizza siciliana              246000
100      bandeja paisa                208800
200      lasagna                      155000
```

Condiciones de la Entrega

1. Fecha

La entrega se hace a través de la actividad establecida para tal fin en BrightSpace**.**

- Entregas posteriores a esta fecha no serán tenidas en cuenta.
- Recuerde que ese mismo día se sustentará, siguiendo las reglas de juego de la asignatura, en la sala de cómputo.

2. Grupos

La entrega se realizará en grupos de trabajo de ***máximo 2 personas***. Los grupos no podrán cambiar su conformación y desde el comienzo dichos grupos estarán identificados plenamente.

3. Entregables

- Un solo archivo .zip nombrado de la siguiente manera: **`PA-Proy1-Nombre1Apellido1-Nombre2Apellido2.zip`** en donde `Nombre1Apellido1` es el nombre y apellido del primer integrante del grupo, y `Nombre2Apellido2` es el nombre y apellido del segundo integrante del grupo. Dentro de este archivo comprimido deben ir los siguientes archivos:
    - Archivo `restaurante.zip` con el código fuente y ejecutable.

4. Observaciones

- Si no se entregan los archivos enumerados previamente, no se calificará la entrega.
- Se reducirán puntos por malas prácticas de programación:
    - Código "quemado" no autorizado. Por ejemplo, usar valores constantes en donde no se deba.
    - Código que no compile o no ejecute, tendrá el proyecto como nota 0.0/5.0

- SUSTENTACION INDIVIDUAL: en caso de no ser exitosa la sustentación, se reconocerá sólo el 30% del total de puntos obtenidos en la entrega.
- Aunque el proyecto puede ser desarrollado en cualquier IDE, éste debe funcionar en las máquinas de la universidad. Se probará la compilación y ejecución por consola. Igual para la sustentación del proyecto.
- Se deben cumplir las Reglas de Juego del curso.